Mobile Systems Development

COMP826

A mobile-based diet assistant app via CGM Integration

Milestone 1: Scoping Study and System Design

Dr. Nathan Allen

Dr. Matthew Kuo

Claude Niu
ID:22184809
Date:21/08/2026

1 Research Topic, Scope and Role....3
1.1 Research Topic....3
1.2 Key Requirements....3
1.2.1 Functional Requirements....3
1.2.2 Non-Functional Requirements....4
1.3 Scope....4
1.3.1 In Scope....4
1.3.2 Out of Scope....4
1.4 Role....5
2 Development process....5
2.1 Waterfall....5
2.2 Scrum....5
2.3 Kanban....6
2.4 Selection of development process....7
2.5 Gantt Chart and Development Process....7
3 Technology selection....8
3.1 React Native....8
3.2 Flutter....8
3.3 Native Android with Kotlin....9
3.4 Selection of Technology....9
4 Logical decomposition....10
4.1 Key Considerations....10
4.2 Choosing architectural pattern....10
4.3 Component diagram....11
4.4 Class diagram....12
5 Runtime (Process) Characteristics....12
5.1 Key Considerations....12
5.2 Overall System Functionality....13
5.3 Details of Runtime Process....14
6 Physical Characteristics....15
6.1 Key Considerations....15
6.2 Overall System Configuration....16
7 Evaluation Criteria....17
7.1 Key Considerations....17
7.2 Detailed Evaluation Criteria....17
8 References:....19

1 Research Topic, Scope and Role

1.1 Research Topic

The traditional approach to mobile dietary management—relying on static caloric counting and standardized nutritional databases (e.g., MyFitnessPal)—often fails to account for the high degree of inter-individual variability in metabolic responses to identical foods (Zeevi et al., 2015). This project aims to address this limitation by developing a mobile system that provides personalized physiological feedback.

The system uses Continuous Glucose Monitor (CGM) data, which is increasingly applied beyond diabetes management in personalized nutrition (Pullano et al., 2022). Its backend calculates postprandial incremental Area Under the Curve (iAUC) from glucose time-series data using the Trapezoidal rule, providing a validated measure of glycaemic response. The iAUC provides a quantitative measure of the incremental postprandial glycaemic response to a meal (Ong et al., 2026).

The mobile frontend also uses computer vision to identify and log food from meal images, reducing reliance on manual input (Kalaimani et al., 2026). The identified food information and calculated iAUC are then processed by an LLM to generate personalized dietary feedback, reflecting the growing use of generative AI for synthesizing health data and supporting tailored interventions (Abdur Rahman et al., 2026). BioSync therefore integrates CGM data, mathematical processing, computer vision, and generative AI within a single mobile system.

Several commercial systems, including Levels and January AI, already combine CGM data with meal tracking and personalized metabolic feedback (January AI, 2026; Levels, 2026). However, their analysis is often presented through proprietary scores or prediction models. BioSync instead uses a transparent pipeline in which CGM data is first converted into iAUC using deterministic numerical integration, then combined with food information for LLM-based interpretation.

1.2 Key Requirements

1.2.1 Functional Requirements

- Health Data Synchronization: Retrieve timestamped glucose readings from a supported health-data source such as HealthKit or Health Connect.
- Food Logging: Allow users to capture meal images and use a Computer Vision API to identify food items.

- Metabolic Calculation: Calculate postprandial incremental Area Under the Curve (iAUC) from CGM data using the Trapezoidal rule.
- Dietary Feedback: Combine food information and iAUC results and submit them to an LLM API to generate personalized, non-diagnostic dietary advice.
- Data Visualization: Display glucose trends, meal events, and the associated postprandial response.

1.2.2 Non-Functional Requirements

- Usability: The main meal-logging workflow should require no more than four primary user actions, with glucose charts supporting responsive zooming and panning.
- Performance: The iAUC calculation for up to 500 glucose readings should complete within 500 ms, excluding network and external API latency.
- Reliability: Temporary network loss should not result in the loss of previously retrieved glucose data; locally cached records should synchronize when connectivity returns.
- Security and Privacy: Health-data access requires explicit user permission, all client-server communication must use HTTPS, and external API credentials must remain on the backend.
- Maintainability: iAUC calculation, Computer Vision integration, and LLM integration should be implemented as separate modules.
- Scalability: The serverless backend should automatically scale to support concurrent requests.

1.3 Scope

1.3.1 In Scope

Mobile application

- Retrieve or import CGM glucose data.
- Capture and analyze meal images.
- Display glucose trends, iAUC results, and personalized dietary feedback.
- Cache data during temporary network loss.

Backend

- Process glucose and meal data.
- Calculate postprandial iAUC.
- Integrate Computer Vision and LLM services.
- Return analysis results through a serverless API.

1.3.2 Out of Scope

- Direct BLE communication or reverse engineering of CGM devices.
- Custom AI model training or CGM hardware development.
- Medical diagnosis, medication, or insulin-dosing recommendations.
- Production-scale clinical or regulatory infrastructure.

1.4 Role

A complete BioSync project would typically involve the following roles:

- **Project Manager / Product Owner:** Defines the project direction, manages priorities, and coordinates external services and resources.
- **UI/UX Designer:** Designs an intuitive interface and clear visualisation of glucose and dietary information.
- **Mobile Frontend Developer:** Develops the React Native application and integrates mobile features such as the camera and health-data access.
- **Backend & Algorithm Engineer:** Implements the cloud backend, iAUC calculation, and external Computer Vision and LLM integrations.
- **Quality Assurance Engineer:** Designs and performs tests for functionality, reliability, and integration.

A multidisciplinary approach is important in mHealth development because technical architecture, data integration, and user-centred design can strongly affect system success. (Noorbergen et al., 2021)

For this individual project, I will primarily act as the Mobile Frontend Developer and Backend & Algorithm Engineer, with emphasis on implementing the CGM processing, iAUC calculation, LLM integration, and a basic mobile interface to demonstrate the complete workflow.

2 Development process

2.1 Waterfall

Waterfall is a linear development process in which requirements, design, implementation, testing, and deployment are completed sequentially. It provides clear documentation and milestones, making it suitable for stable requirements. However, changes are difficult once a phase is completed, reducing its suitability for mobile projects where requirements may evolve (Thesing et al., 2021).

2.2 Scrum

Scrum is a structured Agile framework that delivers products through fixed-length Sprints, typically lasting 1–4 weeks. It defines specific roles and ceremonies, including Sprint Planning, Daily Stand-ups, and Retrospectives, enabling frequent feedback and backlog adjustment. However, its administrative overhead and rigid role structure can be inefficient for small teams or lightweight projects (Cho, n.d.).

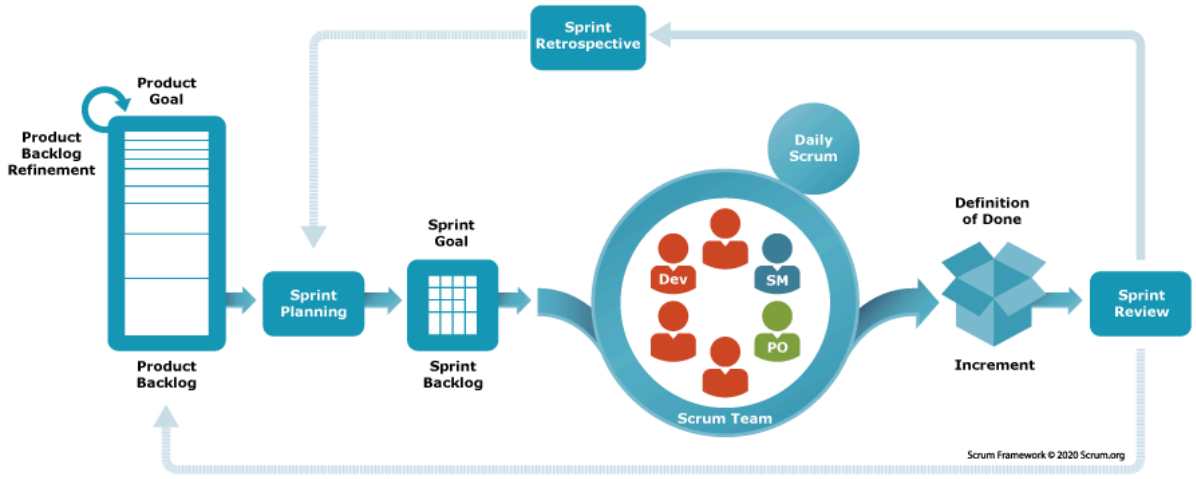


Figure 1.1. Scrum Framework (Scrum.org, n.d.)

2.3 Kanban

Kanban is a lean Agile methodology focused on visualizing workflow, limiting Work in Progress (WIP), and maximizing continuous delivery. Unlike Scrum, Kanban does not operate on fixed-length iterations; instead, tasks are continuously pulled from a backlog into active development columns on a visual board as capacity allows. This approach helps identify bottlenecks and provides substantial flexibility to shift priorities at any moment. However, because Kanban lacks the structured cadences and defined planning phases of Scrum, it can sometimes lead to a lack of predictable milestone forecasting, which is often required in academic or strictly time-boxed projects (Matharu et al., 2015).

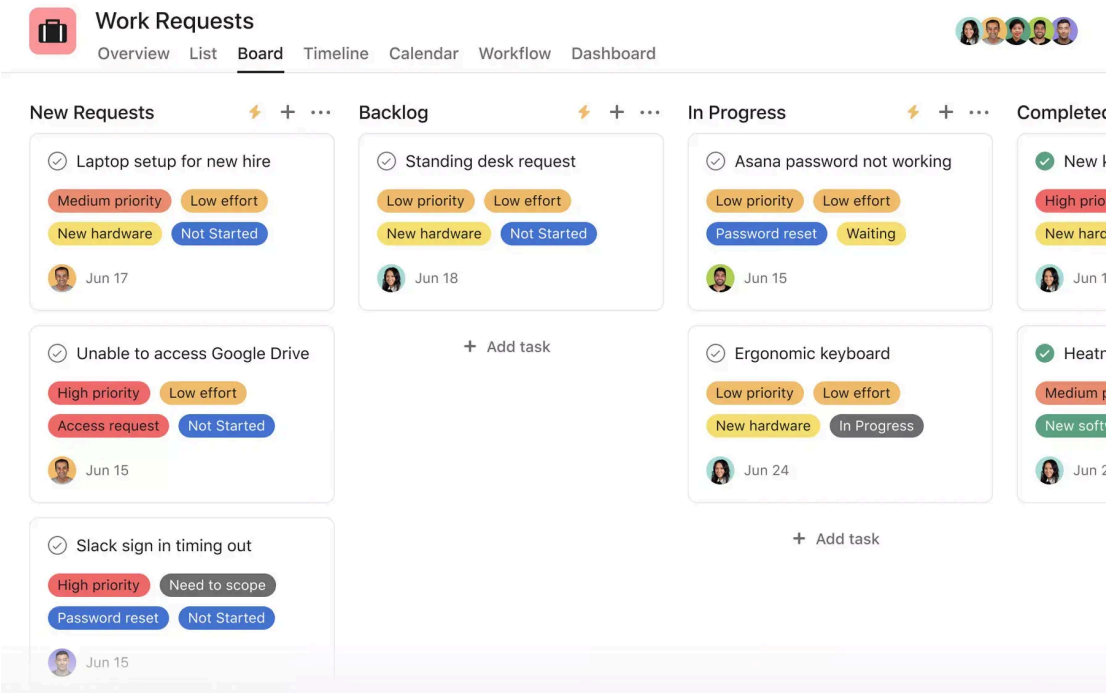


Figure 1.2. Kanban Board (Asana, n.d.)

2.4 Selection of development process

The BioSync project will primarily follow a Kanban-based development process (like Scrumban but not exactly the same), as its visual workflow and continuous task prioritization are well suited to an individually developed mobile project. Tasks will move through stages such as *To Do, In Progress, Testing,* and *Done*, with Work in Progress (WIP) limits used to reduce unfinished work.

To balance Kanban's flexibility with more predictable progress, selected Scrum practices will be incorporated through periodic planning and review checkpoints. Those Scrum practices will be incorporated through weekly planning and review, while daily development tasks will be managed using a Kanban board. For example, check this week's tasks every Monday and what was done last week. This combination retains Kanban's flexibility while providing regular opportunities to evaluate progress and adjust priorities before the final delivery.

2.5 Gantt Chart and Development Process

Figure 2.1 — BioSync Development Gantt Chart

Development Activity	Start	End	Break 24–31 Aug	Week 7 1–6 Sep	Week 8 7–13 Sep	Week 9 14–20 Sep	Week 10 21–27 Sep	Week 11 28 Sep–4 Oct	Week 12 5–11 Oct
Upskilling & environment setup	24 Aug	31 Aug							
Requirements review & feature design	1 Sep	6 Sep							
Frontend development - **React Native**	1 Sep	20 Sep							
Backend development - Python Flask	7 Sep	20 Sep							
Integration & testing	21 Sep	4 Oct							
Final refinement & report writing	5 Oct	11 Oct							

Figure 2.1 — BioSync Development Gantt Chart

Figure 2.2 — Kanban-based Development Process

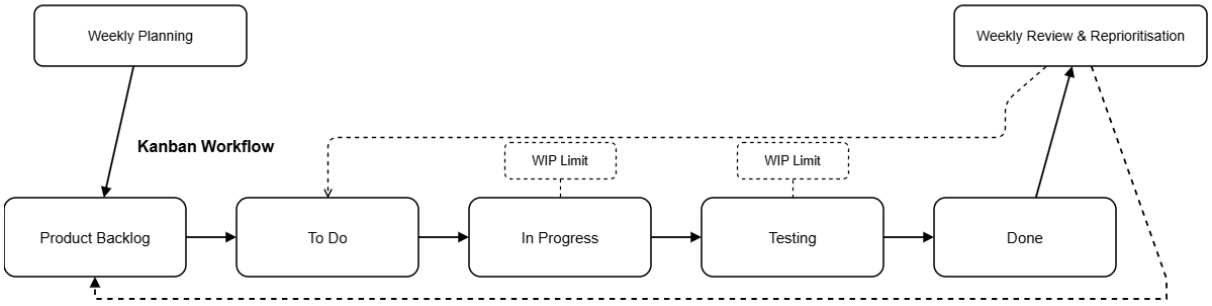


Figure 2.2 — Kanban-based Development Process

Tasks are continuously pulled from the backlog through the Kanban stages. Weekly planning selects and clarifies upcoming work, while weekly review checks completed work and adjusts backlog priorities for the next cycle.

3 Technology selection

BioSync requires access to mobile health data, the device camera, interactive visualization, and cloud APIs. Portability and development time are also important because the proposed system should support both major mobile platforms. Therefore, three commonly used mobile development technologies were considered: React Native, Flutter, and native Android development with Kotlin.

3.1 React Native

React Native is a cross-platform framework using JavaScript or TypeScript that supports substantial code reuse between Android and iOS while retaining access to native platform capabilities (Fentaw, 2020). For BioSync, it provides a suitable balance between portability and native integration for health-data access, camera input, visualisation, and backend communication. Platform-specific services such as Health Connect and HealthKit may still require native modules or compatible libraries.

3.2 Flutter

Flutter is a cross-platform framework using Dart and a shared codebase for Android and iOS. Its own rendering system supports consistent interfaces across platforms (Fentaw, 2020). Flutter would also support BioSync's glucose visualisation and mobile interface, but platform-specific health integration may require plugins or platform channels. Compared with

React Native, it would additionally require adopting Dart and a separate development ecosystem.

3.3 Native Android with Kotlin

Kotlin is a statically typed language widely used for native Android application development. Native development provides direct access to Android platform APIs and Jetpack libraries, making it well suited to applications requiring close integration with Android services such as Health Connect. Kotlin has also been studied as a language for developing production Android applications (Góis Mateus & Martinez, 2019).

For BioSync, Kotlin would provide straightforward access to Android health APIs and strong native performance. However, it is platform-specific; supporting iOS would require a separate implementation using technologies such as Swift. This would increase development and maintenance effort compared with a shared cross-platform codebase.

3.4 Selection of Technology

React Native was selected for BioSync because it provides cross-platform development while retaining access to native mobile capabilities. Compared with Flutter, it avoids introducing an additional Dart-based development stack, while compared with native Kotlin it provides significantly greater code reuse across Android and iOS. Its potential limitations for heavy local computation are less significant for BioSync because iAUC calculation and AI orchestration are performed by the backend.

The proposed technology stack is summarised below.

Category	Selected Technology	Purpose
Mobile Framework	React Native	Cross-platform mobile application
Mobile Language	TypeScript / JavaScript	Frontend development
Architecture	MVVM	Separation of presentation and application logic
Health Integration	Health Connect / HealthKit	Access to glucose and health data
Backend	Python + Flask	API orchestration and iAUC processing
Cloud Platform	AWS Lambda + API Gateway	Serverless backend deployment

Category	Selected Technology	Purpose
Mobile Framework	React Native	Cross-platform mobile application
Mobile Language	TypeScript / JavaScript	Frontend development
Architecture	MVVM	Separation of presentation and application logic
AI Services	Computer Vision API + LLM API	Food recognition and dietary feedback
Local Storage	SQLite	Temporary offline data caching
Communication	REST over HTTPS	Secure client-server communication

4 Logical decomposition

4.1 Key Considerations

The logical design is guided primarily by the functional requirements for health-data retrieval, meal logging, metabolic analysis, dietary feedback, and data visualization. To maintain separation of concerns, the mobile application is decomposed into presentation, application-state, and data-management responsibilities. The design should also isolate health, meal, and analysis data access so that individual services can be modified without requiring major changes to the user interface.

4.2 Choosing architectural pattern

Selecting the correct global architecture is crucial for separating application functionality and promoting organized programming. Common patterns used in the industry include MVC, MVP, MVVM, and VIPER.

- **MVC (Model View Controller):** This is one of the most frequently used patterns where the View has direct access to the Model. While it separates functionality, it is noted to work better specifically for native Android development.
- **MVP (Model View Presenter):** In this pattern, the Model and View are loosely coupled, and interfaces are used to pass information between them. This approach works well for both iOS and Android platforms.
- **VIPER (View-Interactor-Presenter-Entity-Router):** This pattern isolates navigational logic within a dedicated Router module, providing better scalability and maintainability. It is highly suited for larger projects.
- **MVVM (Model View ViewModel):** Emerging from MVC and MVP, MVVM utilizes direct data bindings between the view and the viewmodel, which reduces the need

> for interfaces. Furthermore, the view is only exposed to a subset of the entire model, making it an efficient choice for smaller projects.
>
> (Dobrean & Dioșan, 2019)

I will use the MVVM architectural pattern for the React Native mobile client. MVVM was selected for the React Native client because it separates UI presentation state from application logic and data-access responsibilities. This structure aligns with BioSync's repository-based design, reducing coupling between the interface and the underlying health, meal, and analysis data sources while avoiding the additional structural complexity of VIPER.

4.3 Component diagram

Figure 4.1 — BioSync Component Diagram

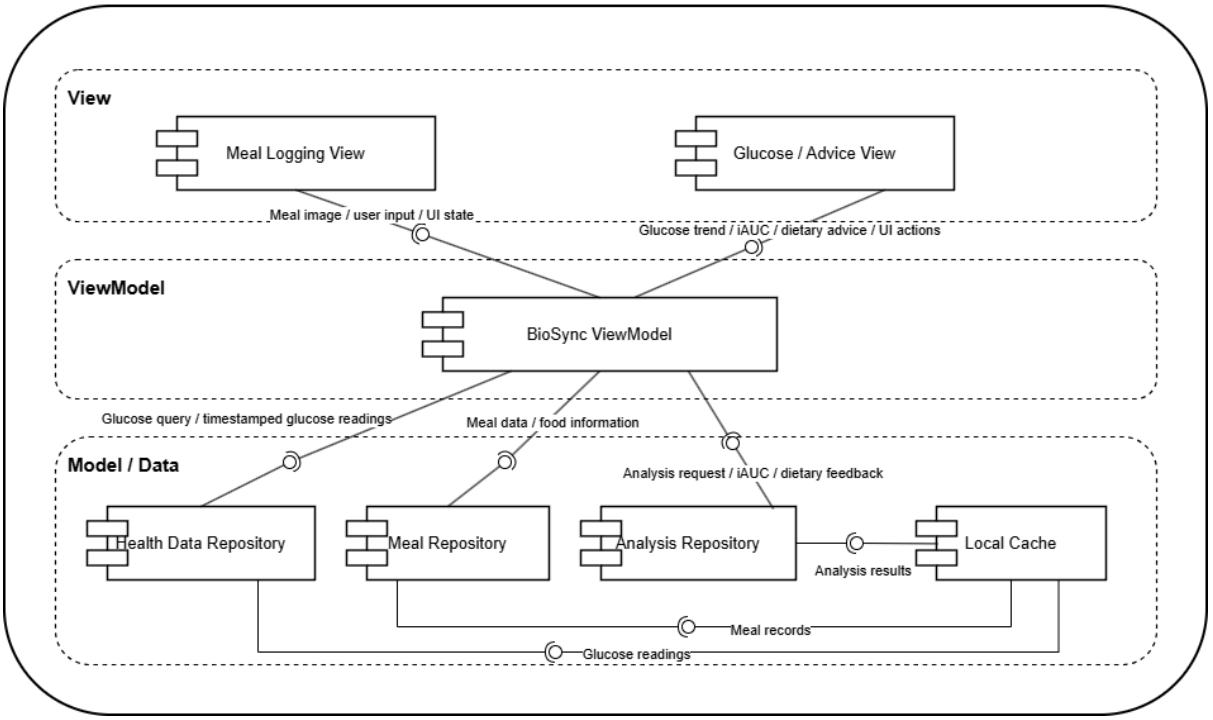


Figure 4.1 – BioSync Component Diagram

Figure 4.1 presents the logical components of the BioSync mobile application using the MVVM architecture. The View layer contains the meal logging and glucose/advice interfaces, while the BioSync ViewModel manages presentation state and coordinates application operations. The Model/Data layer consists of repositories responsible for health, meal, and analysis data, together with a local cache for temporary data storage. This separation reduces coupling between the user interface and data-management logic.

4.4 Class diagram

Figure 4.2 — BioSync Class Diagram

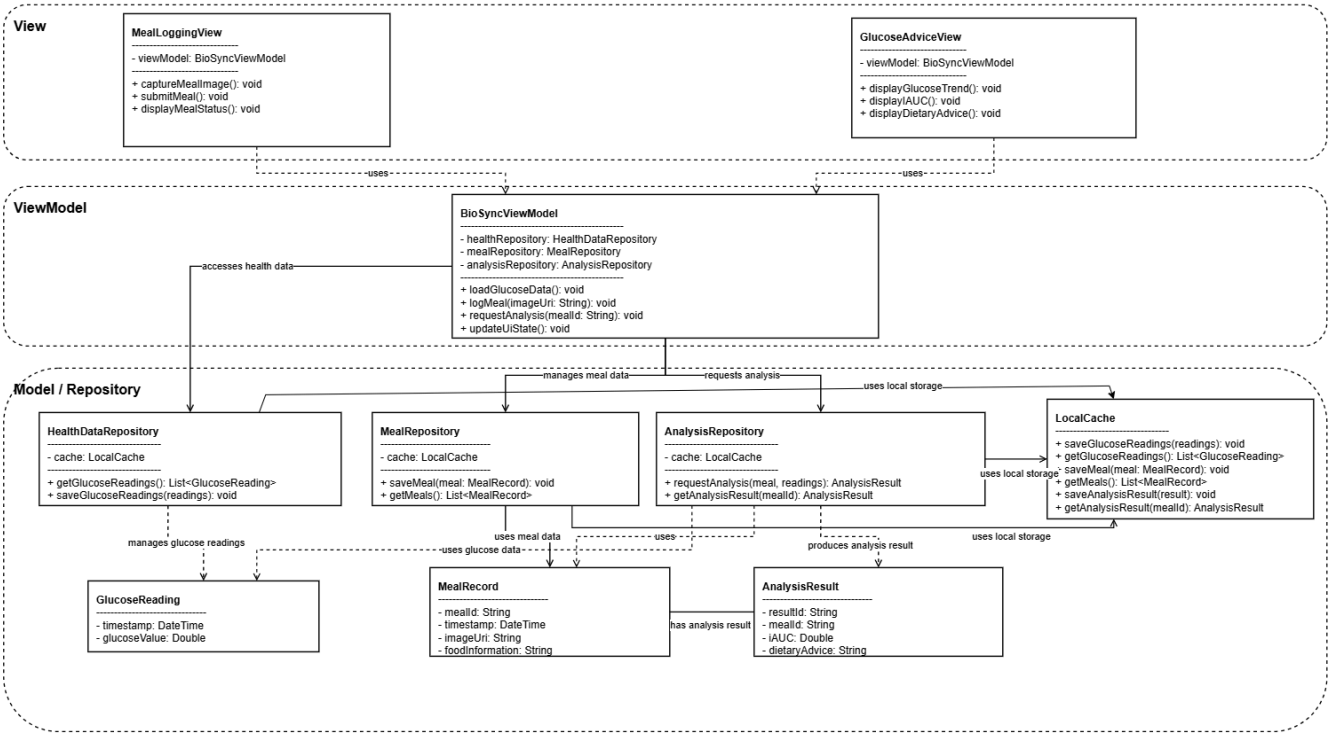


Figure 4.2 further refines the MVVM components into their main classes and responsibilities. The View classes interact with BioSyncViewModel, which accesses the health, meal, and analysis repositories. These repositories manage domain objects such as GlucoseReading, MealRecord, and AnalysisResult, while LocalCache provides local persistence. The class structure maintains separation between presentation, application logic, and data access, supporting maintainability and future modification.

5 Runtime (Process) Characteristics

5.1 Key Considerations

The runtime design of BioSync is primarily guided by usability, responsiveness, network reliability, and resource efficiency. Mobile applications operate under constraints including limited battery capacity, variable connectivity, network usage, and memory consumption.

Usability is particularly important because meal logging is expected to be a repeated daily activity. The PACMAD usability model identifies effectiveness, efficiency, satisfaction, learnability, memorability, errors, and cognitive load as important attributes of mobile usability (Harrison et al., 2013). For BioSync, efficiency and cognitive load are prioritised by reducing manual food entry and presenting glucose response and dietary feedback through a simple workflow.

The main runtime requirements are:

Quality	Runtime Requirement / Design Response
Usability	The primary meal-logging workflow should require no more than four main user actions from starting meal capture to submitting the meal for analysis.
Responsiveness	iAUC calculation for up to 500 glucose readings should complete within 500 ms, excluding network and external API latency. External AI requests execute asynchronously so that the mobile UI remains responsive.
Reliability	Retrieved glucose, meal, and analysis data are temporarily cached locally so that temporary network interruption does not remove previously available information.
Network / Power	Only glucose data relevant to the analysis period are retrieved and meal images are uploaded only when analysis is requested, avoiding unnecessary continuous network activity.

5.2 Overall System Functionality

BioSync is designed around a short navigation path for its main activities. From the main dashboard, users can access meal logging, glucose trends, and previously generated analysis results. Meal logging guides the user from image capture to confirmation and analysis without requiring manual navigation through unrelated screens.

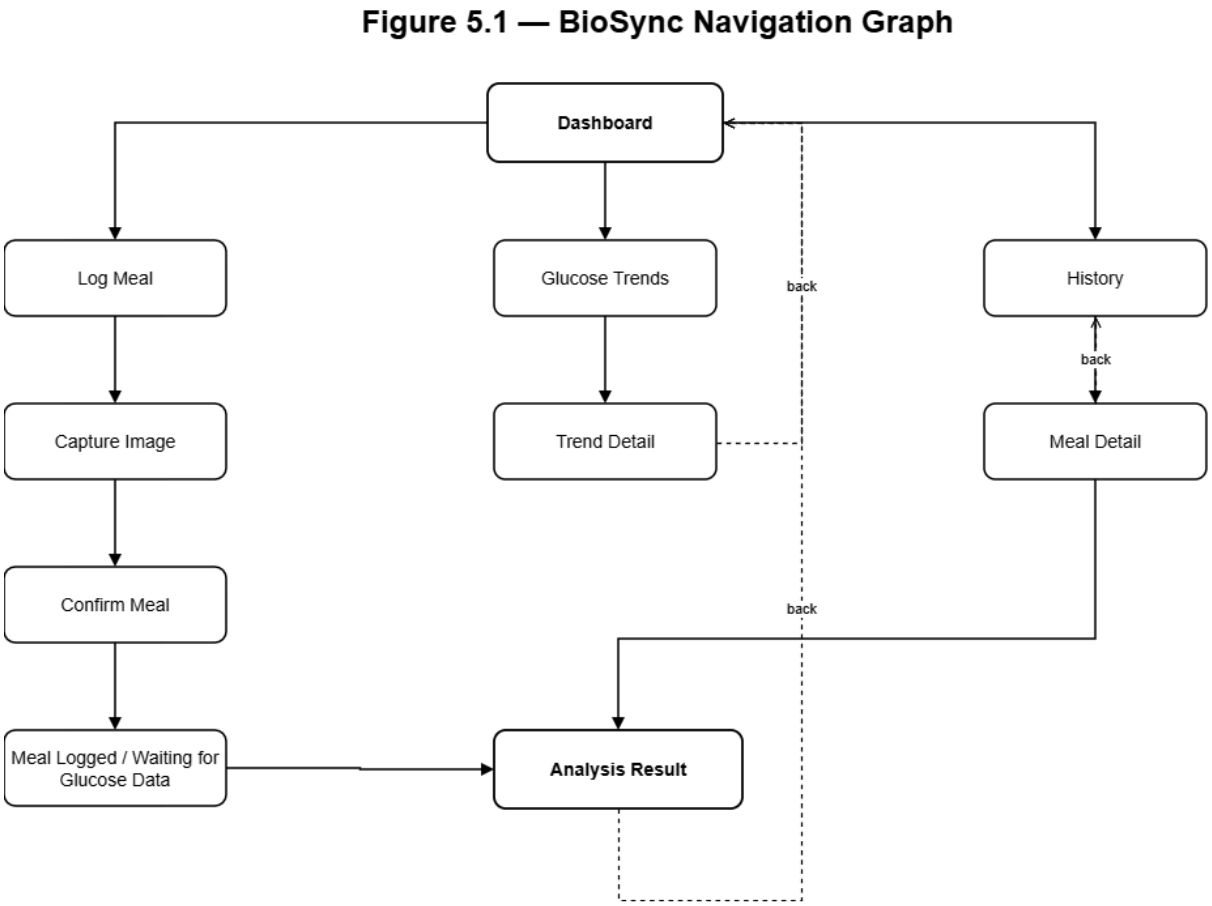


Figure 5.1 —BioSync Navigation Graph

Figure 5.1 shows the primary navigation paths of BioSync. The core meal-analysis workflow follows a short linear path from meal capture to metabolic feedback, while glucose trends and historical results remain directly accessible from the dashboard. This structure is intended to reduce navigation complexity and cognitive load.

5.3 Details of Runtime Process

The runtime process begins when the user logs a meal and sends an image to the Computer Vision service, which identifies the food. After the postprandial monitoring period, BioSync retrieves the glucose readings associated with that meal and sends them to the backend for iAUC calculation. The food information and iAUC result are then combined and submitted to the LLM. The generated dietary feedback is returned to the mobile application, cached locally, and displayed with the corresponding glucose response.

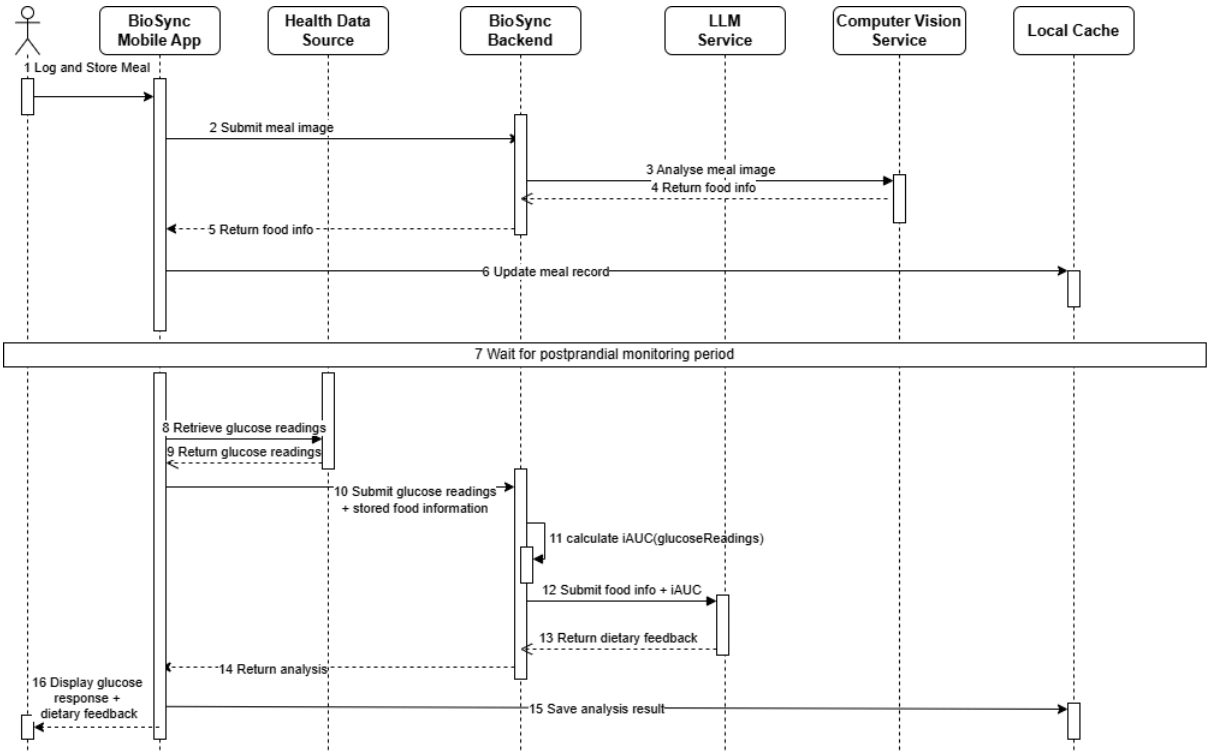


Figure 5.2 — BioSync Runtime Sequence Diagram

6 Physical Characteristics

6.1 Key Considerations

The physical design of BioSync is primarily guided by portability, reliability, security/privacy, and performance. The system separates mobile interaction and local data storage from cloud-based computation. The React Native application executes on an Android or iOS smartphone, where it accesses the device camera, the supported health-data platform, and a SQLite local cache. Computational tasks including iAUC calculation and AI-service orchestration are executed in the cloud through AWS Lambda behind API Gateway.

This separation reduces processing demand on the mobile device while allowing the serverless backend to scale independently. Temporary local caching also allows previously retrieved glucose and meal data to remain available during short network interruptions. Sensitive API credentials are retained on the backend, and communication between the mobile application, backend, and external services uses HTTPS.

The key physical considerations are:

Concern	Physical Design Response

Portability	React Native supports deployment to Android and iOS, while platform-specific health access is provided through Health Connect or HealthKit.
Reliability	SQLite stores temporary local data so previously retrieved records remain available during temporary loss of connectivity.
Security / Privacy	Health-data access requires OS permission; external API keys remain on the backend; network communication uses HTTPS.
Performance / Scalability	iAUC calculation and AI orchestration execute on AWS Lambda rather than the mobile device, reducing device-side processing and allowing backend execution to scale with requests.

6.2 Overall System Configuration

Figure 6.1 shows the deployment of BioSync across the mobile device, cloud backend, and external services. The smartphone hosts the React Native application, local SQLite cache, camera access, and the platform health-data service. Where a compatible source is available, timestamped glucose readings are obtained through Health Connect or HealthKit rather than direct communication with CGM hardware.

The mobile application communicates with Amazon API Gateway over HTTPS. Requests are forwarded to the AWS Lambda backend running the Python/Flask processing logic, where glucose data are processed to calculate iAUC and external AI services are orchestrated. The backend communicates independently with the Computer Vision API and LLM API, then returns the completed analysis to the mobile application for display and local caching.

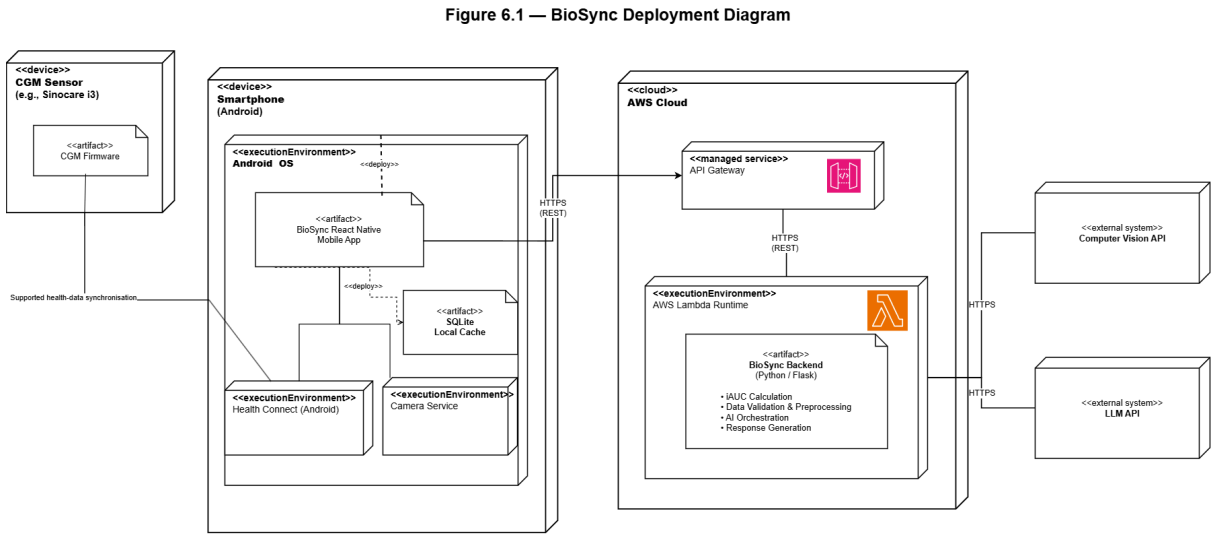


Figure 6.1 — BioSync Deployment Diagram

7 Evaluation Criteria

7.1 Key Considerations

The evaluation of BioSync will focus on the implemented end-to-end mobile workflow rather than evaluating isolated components only. The primary criteria are functional correctness, performance, usability, reliability, and end-to-end integration. Performance and usability are particularly important runtime characteristics for mobile systems, while the usability evaluation will be guided by the PACMAD model, particularly effectiveness, efficiency, and errors (Harrison et al., 2013).

Evaluation criteria are defined using measurable outcomes and explicit test methods. Automated tests will be used where results can be objectively calculated, while task-based user testing will be used for usability characteristics.

7.2 Detailed Evaluation Criteria

Criterion	Measurement	Pass Criterion
iAUC correctness	Compare backend output with manually calculated trapezoidal-rule values.	Error ≤ 1%.

Performance	Measure iAUC processing time for up to 500 glucose readings using backend timing logs.	≤ 500 ms, excluding network/API latency.
Mobile responsiveness	Use Android Studio Profiler / ADB frame statistics during the core workflow and record UI frame behaviour and application responsiveness.	No ANR or frozen frames during the tested workflow.
Usability	PACMAD-based task testing of meal capture, confirmation, and analysis; record actions and errors.	Complete without assistance in ≤ 4 primary actions and no critical errors.
Reliability	Disable network using ADB after caching data, then reopen the relevant screen.	Cached data remains accessible without crash or loss.
End-to-end integration	Trace one test case from mobile input through CGM data, backend, CV/iAUC, LLM, and result display.	Complete result returned with no missing or duplicated data.

8 References:

Abdur Rahman, L., Dedousis, V., Papathanail, I., Poursoleymani, R., Kafyra, M., Kalafati, I. P., & Mougiakakou, S. G. (2026). Generative AI in Precision Nutrition: A Review of Current Developments and Future Directions. *Nutrients*, *18*(6), 938. https://doi.org/10.3390/nu18060938

Asana. (n.d.). *Kanban Explained: Guide to Agile Boards, Cards & WIP Limits [2026] • Asana*. Asana. Retrieved August 21, 2026, from https://asana.com/resources/what-is-kanban

Cho, J. J. (n.d.). *An Exploratory Study on Issues and Challenges of Agile Software Development with Scrum*.

Dobrean, D., & Dioșan, L. (2019). A Comparative Study of Software Architectures in Mobile Applications. *Studia Universitatis Babeș-Bolyai Informatica*, *64*(2), 49–64. https://doi.org/10.24193/subbi.2019.2.04

Fentaw, A. E. (2020). *Cross platform mobile application development: A comparison study of React Native Vs Flutter*.

Góis Mateus, B., & Martinez, M. (2019). An empirical study on quality of Android applications written in Kotlin language. *Empirical Software Engineering*, *24*(6), 3356–3393. https://doi.org/10.1007/s10664-019-09727-4

Harrison, R., Flood, D., & Duce, D. (2013). Usability of mobile applications: Literature review and rationale for a new usability model. *Journal of Interaction Science*, *1*(1), 1. https://doi.org/10.1186/2194-0827-1-1

January AI. (2026). *Platform—January AI Health Context Engine*. January AI. https://january.ai/platform

Kalaimani, M., D, A. S., M, D. S., & Santhoshkumar, S. P. (2026). Smart Food Recognition and Nutrition Estimation using Computer Vision. *2026 9th International Conference on Inventive Computation Technologies (ICICT)*, 1198–1204. https://doi.org/10.1109/ICICT68280.2026.11510899

Levels. (2026). *Levels—Live healthier, longer*. https://www.levels.com/#what-you-get

Matharu, G. S., Mishra, A., Singh, H., & Upadhyay, P. (2015). Empirical Study of Agile Software Development Methodologies: A Comparative Analysis. *ACM SIGSOFT Software Engineering Notes*, *40*(1), 1–6. https://doi.org/10.1145/2693208.2693233

Noorbergen, T. J., Adam, M. T. P., Roxburgh, M., & Teubner, T. (2021). Co-design in mHealth Systems Development: Insights From a Systematic Literature Review. *AIS Transactions on Human-Computer Interaction*, *13*(2), 175–205. https://doi.org/10.17705/1thci.00147

Ong, L., Lamoth, C. J., van Beek, A., Cao, M., Verkerke, G. J., & Wilhelm, E. (2026). Continuous Glucose Measurements for Diet Monitoring in Healthy Adults. *Journal of Diabetes Science and Technology*, *20*(4), 1242–1251. https://doi.org/10.1177/19322968251361555

Pullano, S. A., Greco, M., Bianco, M. G., Foti, D., Brunetti, A., & Fiorillo, A. S. (2022). Glucose biosensors in clinical practice: Principles, limits and perspectives of currently used devices. *Theranostics*, *12*(2), 493–511. https://doi.org/10.7150/thno.64035

Scrum.org. (n.d.). *What is Scrum?* Retrieved August 21, 2026, from https://www.scrum.org/resources/what-scrum-module

Thesing, T., Feldmann, C., & Burchardt, M. (2021). Agile versus Waterfall Project Management: Decision Model for Selecting the Appropriate Approach to a Project. *Procedia Computer Science, CENTERIS 2020 - International Conference on ENTERprise Information Systems / ProjMAN 2020 - International Conference on Project MANagement / HCist 2020 - International Conference on Health and Social Care Information Systems and Technologies 2020, CENTERIS/ProjMAN/HCist 2020*, *181*, 746–756. https://doi.org/10.1016/j.procs.2021.01.227

Zeevi, D., Korem, T., Zmora, N., Israeli, D., Rothschild, D., Weinberger, A., Ben-Yacov, O., Lador, D., Avnit-Sagi, T., Lotan-Pompan, M., Suez, J., Mahdi, J. A., Matot, E., Malka, G., Kosower, N., Rein, M., Zilberman-Schapira, G., Dohnalová, L., Pevsner-Fischer, M., … Segal, E. (2015). Personalized Nutrition by Prediction of Glycemic Responses. *Cell*, *163*(5), 1079–1094. https://doi.org/10.1016/j.cell.2015.11.001

Word count: 3880